

Hyperparameter Optimization of a Neural Network for Hotel Customer Feedback Classification

John Laurence E. Ong
College of Computer Science and
Engineering
Jo Rizal University
Mandaluyong City, Philippines
johnlaurence.ong@my.jru.edu

Abstract—This exercise evaluates the baseline accuracy from the previous exercise using the provided Keras model. A baseline no-code model created in PT-P1 was used for comparison. Which achieved a weighted F1-score of 83.98%, was used as the basis for comparison. The hotel feedback dataset in this exercise was preprocessed through vocabulary construction, converting words into numbers, padding, and truncation, and was divided into 70% training, 15% validation, and 15% testing sets. Using TensorFlow and the Keras API, a Sequential model consisting of Embedding, GlobalAveragePooling1D, Dense, Dropout, and Softmax output layers was trained through a set of experiments involving changes in learning rate, hidden-layer size, dropout rate, batch size, epochs, and dataset expansion and added early stopping. Training and validation loss curves were also monitored to identify underfitting and overfitting, goodfit. The best-performing configuration, EXP-22, used 550 data samples and achieved 98.78% validation accuracy and a 98.78% weighted F1-score. The findings show that configuration hyperparameter, early stopping, and increased training data significantly improved model generalization compared with the PT-P1 baseline.

Keywords—Neural Network, Hotel Customer Feedback, Sentiment and Intent Classification, TensorFlow, Keras, Hyperparameter Tuning, Weighted F1-Score, Early Stopping.

I. INTRODUCTION

Hotel customer feedback may contain experiences, opinions, and concerns that can be understood by reviewing the feedback. However, manually reviewing a lot of customer feedback can be time consuming. Sentiment analysis can help process text and identify the opinions shared by customers.[1] Used sentiment analysis on hotel reviews and showed that neural network models can be used to classify customer opinions. [2] Applied a Keras Sequential neural network model approach to hotel reviews to identify customer sentiment and preferences. That model learns patterns from training data and uses these patterns to

classify new data. How well the model can perform well on data that was not used during training is important for model generalization [3]. In this exercise, a Keras Sequential architecture was provided to classify hotel customer feedback based on sentiment and intent. Different training settings were tested to determine which configuration could provide better classification performance.

The goal of this exercise was to train and improve a Keras Sequential neural network model for hotel customer feedback classification. The model was provided in the google colab implemented in Python using TensorFlow and Keras. The dataset was divided into 70% training, 15% validation, and 15% testing data. Sets of experiments were tested by added regularization and changing the configuration hyperparameters settings. These included the learning rate, dropout rate, batch size, epochs, and optimizer, early stopping. Training and validation loss were also observed to identify underfitting, overfitting, and good fit. The goal was to find an optimal configuration that could beat the PT-P1 no-code baseline.

The PT-P1 no-code previous exercise was used as the basis for comparing the results of the following neural network experiments. Based on the monitoring log the PT-P1 exercised a precision of 81.82%, recall of 88.89%, and an F1-score of 83.98%. This results served as the basis for determining whether the Keras Sequential neural network model improved the classification performance. Unlike the PT-P1 no-code approach, the PT-P2 exercise allowed changes to the model and the configuration settings of the provided hyperparameters. Different hyperparameter values were tested, and the training and validation results were logs for each experiment. The results were then compared with the PT-P1 baseline to determine which configuration provided better performance.

II. EXPERIMENTAL METHODOLOGY AND HYPERPARAMETER CONFIGURATION

A. Dataset Preprocessing

The dataset used in this experiment contained hotel customer feedback labeled based on the sentiment and intent. Before model training, the text data was prepared so that it could remove the unnecessary text to apply in the model. A vocabulary was created from the feedback text with an initial maximum vocabulary size of 1,000 words. Two values were reserved, where 0 represented padding (PAD) and 1 represented unknown words (UNK). Each customer feedback was converted to lowercase and separated into words. The words were then converted into numerical sequences based on their assigned values in the vocabulary. Words that were not included in the vocabulary were assigned the UNK value. Padding and truncation were also applied using an initial maximum sequence length of 20 tokens to provide the same input length for each feedback. For the later experiments, additional training data and changes in the configuration settings to improve model performance.

B. Data Partitioning

After preprocessing, the dataset was divided into training, validation, and testing sets as needed in the exercise. The training set contained 70% of the data and was used to train the neural network and update its parameters. The validation set contained 15% and was used to monitor the model during training and compare the different configurations. The remaining 15% was kept as the test set for evaluating the trained model on unseen data. Also in the provided code a fixed random state was used during data splitting to keep the distribution consistent when the experiment was repeated. Random seeds were also set in NumPy and TensorFlow to provide more consistent model initialization and experimental results.

C. Neural Network Architecture

The provided architecture in the exercise, a TensorFlow/Keras Sequential model, was used for the classification of hotel customer feedback. The model consisted of an Embedding layer, GlobalAveragePooling1D layer, Dense hidden layer with ReLU activation, Dropout layer, and a Dense output layer with Softmax activation. The Embedding layer converted the numerical word values into vector representations. The GlobalAveragePooling1D layer combines these vectors into a single representation of each text feedback. The Dense layer then processed the extracted information, while the Dropout layer temporarily off a portion of the neurons during training to reduce overfitting. Lastly, the Softmax output layer produced the probabilities used to classify the customer feedback into positive, negative, urgent.

D. Hyperparameter Configuration and Experimental Setup

The experiments process by changing different hyperparameters grouped according to the three pillars provided in lecture to apply the exercise. The architecture settings included the embedding dimension and number of hidden neurons. The optimization settings included the learning rate, batch size, number of epochs, and optimizer. The regularization setting included the dropout rate. The initial Keras model used an embedding dimension of 32, 64 hidden neurons, a learning rate of 0.005, a dropout rate of 0.1, a batch size of 32, and 15 epochs. Adam was used to optimize the model, while cross-entropy was used to calculate the classification loss. In the next experiments, these settings were changed to observe their effects on training and validation performance. Early stopping was also added during optimization to monitor validation loss and stop training when the model stopped improving.

In the part of experiments setup the Keras baseline using the initial hyperparameter settings. In the second experiment, the learning rate, hidden-layer size, dropout rate, batch size, epochs, and other settings were changed to look at their effects on model performance. Some configurations were used to observe underfitting and overfitting, while later experiments focused on improving model generalization. When changes in hyperparameters were not enough to reach the target results the amount of training data was expanded and additional configurations were tested. Each experiment was monitored using training loss, training accuracy, validation loss, validation accuracy, and validation weighted F1-score. Training and validation loss curves were also generated to determine whether each configuration showed underfitting, overfitting, or a good fit. The final configuration was selected based mainly on its validation performance and weighted F1-score.

III. PERFORMANCE MONITORING AND LOSS-CURVE ANALYSIS

The performance of each experiment was identified using training loss, training accuracy, validation loss, validation accuracy, and validation weighted F1-score. The training and validation loss curves were also observed to determine whether the model showed underfitting, overfitting, or good fit. Table I. presents the results recorded in the experiment monitoring log.

A. Summary of Model Performance During Experimentation

Model Performance	Experiments	Dataset Sized	Val Accuracy	Weighted F1
Overfitting	EXP-01-EX P-07	50	28.57%–57.14 %	28.63%–57.14 %
Slight Overfitting	EXP-08	200	93.33%	93.16%
Good Fit with Slight Overfit	EXP-09-EX P-18	200–300	90.00%–95.56 %	89.83%–95.54 %
Good Fit / Close to Ideal	EXP-19-EX P-21	550	97.56%	97.56%–97.57 %
Ideal Fit	EXP-22	550	98.78%	98.78%

Table I. Summary of Experimental Model Performance.

Table I. Summary of the configuration hyperparameters change in model to identify the performance throughout the experiments. The early experiments, EXP-01 to EXP-07, used only 50 data samples and mostly showed overfitting with low validation performance. Increasing the dataset to 200 samples in EXP-08 improved the weighted F1-score to 93.16%. And next experiments continued to adjust the learning rate, hidden neurons, dropout rate, batch size, and epochs, resulting in better performance as the dataset was added to 300 and 550 samples. EXP-19 to EXP-21 reached validation accuracy of 97.56%, showing performance close to an ideal fit. Lastly, EXP-22 used Early Stopping and stopped at epoch 20 out of the configured maximum of 45 epochs. It achieved the highest validation accuracy and weighted F1-score of 98.78% and was classified as an ideal fit.

B. EXP-01: Keras Baseline

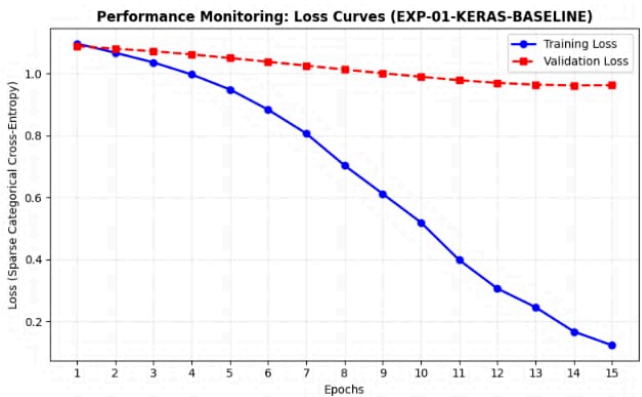


Figure. 1. Training and validation loss of EXP-01 Keras baseline.

EXP-01 was the first in the Keras model baseline. It used the default value of the hyperparameter as 32 hidden neurons, a learning rate of 0.005, a dropout rate of 0.10, a batch size of 32, and 15 epochs. The experiment recorded the training loss of 0.1223 and a validation loss of 0.9628. Training accuracy reached 100%, while validation accuracy and weighted F1-score were only 57.14%. The large difference between the training and validation results showed that the model performed well on the training data but had difficulty with the validation data. The weighted F1-score of 57.14% was also lower than the PT-P1 baseline F1-score of 83.98%. This result showed that the Keras configuration was not enough to outperform the no-code model.

C. EXP-02: Low-Capacity Model

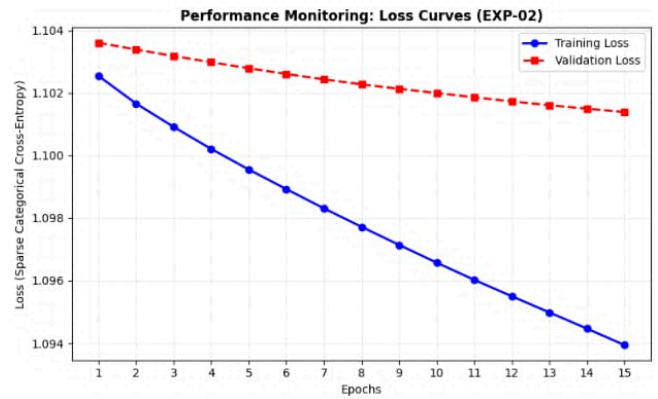


Figure. 1.1. Training and validation loss of EXP-02.

EXP-02 was used with a lower model capacity by reducing the hidden neurons to 16 and the learning rate to 0.0001, while the dropout rate was set to 0. The model recorded a training loss of 1.0939 and a validation loss of 1.1014. It got 57.14% training accuracy, 28.57% validation accuracy, and a weighted F1-score of 28.63%. This results were lower than EXP-01 and the PT-P1 baseline. The small learning rate and lower number of hidden neurons limited the improvement of the model during training. Based on the EXP-02 was recorded as an overfit model.

D. EXP-03: High-Capacity Model

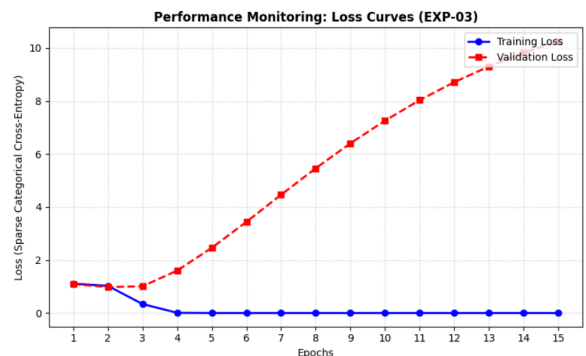


Figure. 1.2. *Training and validation loss of EXP-03.*

EXP-03 tested a higher model capacity by increasing the hidden neurons to 256 and the learning rate to 0.05, without applying dropout. The model got a training loss of 0.0000 and 100% training accuracy. Compared to, the validation loss increased to 12.0200, while the validation accuracy and weighted F1-score were both 57.14%. The large difference between the training and validation loss showed that the model performed very well on the training data but performed worse on the validation data. This showed as overfitting, where the model learned the training data too closely and did not generalize well to the validation data.

E. EXP-04: Regularized Model.

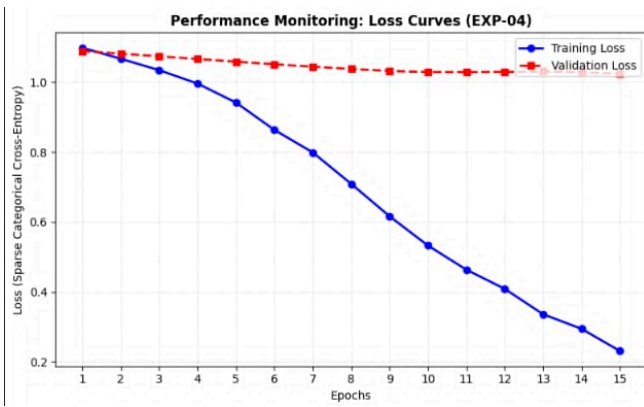


Figure. 1.3. *Training and validation loss of EXP-04.*

EXP-04 was applied to the regularization by applying a dropout rate of 0.40 with 128 hidden neurons and a learning rate of 0.005. The model obtained a training loss of 0.2305 and a validation loss of 1.0237. The training accuracy, validation accuracy, and weighted F1-score were all recorded at 57.14%. The use of dropout provided more prevent overfitting compared with EXP-03. However, the validation performance remained below the PT-P1 baseline F1-score of 83.98%. Based on the monitoring log, the model still showed overfitting. This indicated that changing the hidden neurons and dropout rate alone was not enough to improve performance while the dataset remained at 50 samples.

F. EXP-05 and Custom Optimization Runs.

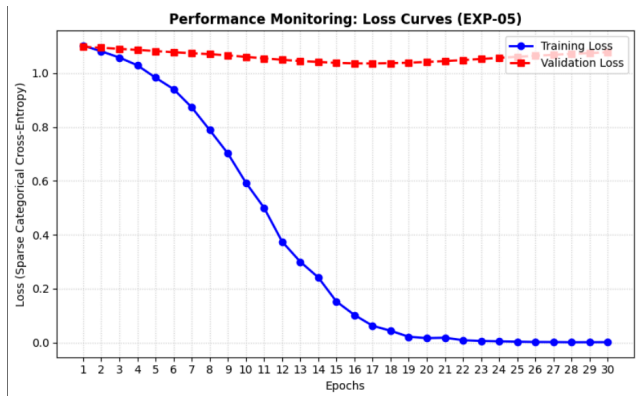


Figure. 1.4. *Training and validation loss during the custom optimization experiments.*

After the first four experiments, additional runs were conducted by changing the learning rate, hidden neurons, dropout rate, batch size, epochs, and amount of training data. EXP-05 to EXP-07 continued to use 50 samples, but their validation accuracy and weighted F1-score remained at 57.14%. These results showed that changing the model settings alone was not enough to reach the PT-P1 baseline.

A larger improvement was observed when the dataset was increased to 200 samples. EXP-08 achieved 93.33% validation accuracy and a weighted F1-score of 93.16%, exceeding the PT-P1 F1-score of 83.98%. EXP-09 to EXP-13 continued to adjust the model settings using 200 samples and generally showed good fit with slight overfitting. The dataset was later increased to 300 samples. EXP-14, EXP-15, EXP-16, and EXP-18 achieved 95.56% validation accuracy and a weighted F1-score of 95.54%.

This showed another improvement compared with the experiments using 200 samples. The last experiments used 550 hotel customer feedback samples. EXP-19 and EXP-20 achieved 97.56% validation accuracy and a weighted F1-score of 97.57%. Further adjustments were made in EXP-21 and EXP-22. In EXP-22, Early Stopping was added while the model was configured for a maximum of 45 epochs. Training stopped at epoch 20, and the model achieved the highest validation accuracy and weighted F1-score of 98.78%. This experiment was recorded as an ideal fit and was selected as the champion model.

G. Champion Model

From the experiment results, EXP-22 reached the highest validation weighted F1-score and was selected as the champion model. Table II. shows the model configuration and its final training and validation performance.

Configuration / Performance	EXP-22
Dataset Size	550
Embedding Dimension	32
Hidden Neurons	32
Learning Rate	0.0008
Dropout Rate	0.62
Batch Size	16
Maximum Epochs	45
Early Stopping Epoch	20
Optimizer	Adam
Training Loss	0.0291
Training Accuracy	100.00%
Validation Loss	0.0273
Validation Accuracy	98.78%
Validation Weighted F1-Score	98.78%
Model Diagnosis	Ideal Fit

Table II. *Training and validation loss of the EXP-22 champion model.*

EXP-22 got a validation accuracy and weighted F1-score of 98.78%, which was higher than the PT-P1 baseline F1-score of 83.98%. The model also recorded a training loss of 0.0291 and a validation loss of 0.0273. These close loss values showed that the model performed well on both the training and validation data.

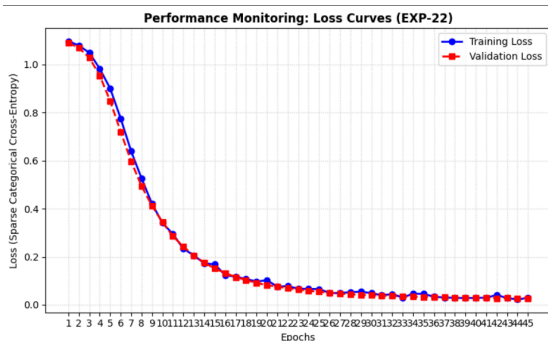


Figure. 1.5 *Training and validation loss curve of the EXP-22 champion model.*

As shown in Fig. 1.5 the training and validation losses decreased during training and stayed close. The model was configured for a maximum of 45 epochs, while Early Stopping ended the training at epoch 20 when the validation loss no longer improved. This helped limit unnecessary training and reduced the chance of further overfitting. The final performance of EXP-22 was achieved through the combination of 550 samples of data and adjustments to the learning rate, hidden neurons, dropout rate, and batch size. With a validation accuracy and weighted F1-score of 98.78%, EXP-22 achieved the highest performance among the experiments and was classified as an ideal fit.

IV. DISCUSSION AND DEBRIEF ANSWERS

A. *Code-Based Optimization vs. No-Code Heuristics (PT-P1 Comparison)*

The Baseline from the previous exercise in no-code has a simpler way to train a model for hotel customer feedback classification because from that baseline it performs the model training automatically. However, the training settings were less adjustable. In other ways, the Keras Sequential neural model implementation gave the training settings to be changed directly in google colab. These included the learning rate, hidden neurons, dropout rate, batch size, epochs, and optimizer. The loss function was also defined in the model configuration, while the optimizer handled the weight updates during training.

This PT2 exercise as code-based approach requires more experiments because changes in the hyperparameters could either improve or reduce model performance. This was observed in EXP-01 to EXP-07, where different configurations still produced validation results below the PT-P1 baseline. However, it is possible to monitor the training and validation loss while adjusting the model settings. The next experiments later increased the weighted F1-score from the PT-P1 baseline of 83.98% to 98.78% in EXP-22. Therefore, the no-code approach was simpler to use, while the code-based approach provided more control over the training process and model improvement.

B. *Diagnosing Model Fit via Loss Curves & Regularization (Pillar 1 & Pillar 3)*

The training and validation loss curves were used to observe the behavior of the model during each

experiment. Based on the provided instruction of this exercise. Overfitting happens when the training loss continues to go down while the validation loss begins to go up or move away from it. Underfitting happens when both losses remain high and the model does not perform well. A good fit happens when both losses go down and remain close to each other.

From that results showed how model capacity affected the model observed in EXP-02 and EXP-03. EXP-02 used only 16 hidden neurons and a learning rate of 0.0001. It recorded a training loss of 1.0939 and validation loss of 1.1014, together with low validation performance. In EXP-03, the hidden neurons were increased to 256 and the learning rate to 0.05 without dropout. Its training loss reached 0.0000, while validation loss increased to 12.0200. This wide gap showed that the model learned the training data closely but did not perform well on the validation data.

Dropout was added in EXP-04 at a rate of 0.40 to provide regularization. However, the model still got only 57.14% validation accuracy and weighted F1-score. This showed that dropout did not provide enough improvement to the model while the amount of training data remained small. In the next experiments, dropout was adjusted together with the other hyperparameters and 550 samples. EXP-22 used a dropout rate of 0.62 and produced close training and validation losses of 0.0291 and 0.0273. Early Stopping was also applied and ended training at epoch 20 instead of continuing to the maximum of 45 epochs. The final loss behavior supported the classification of EXP-22 as an ideal fit.

C. Data Volume & Diversity vs. Architectural Capacity

The sets of experiments in this exercise showed that changing the hyperparameters alone was not enough to outperform the PT-P1 baseline. EXP-01 to EXP-07 used 50 data instances and produced weighted F1-scores ranging from 28.63% to 57.14%. Even when the learning rate, hidden neurons, dropout rate, batch size, and epochs were changed, the validation performance remained limited. A clear improvement was observed after increasing the amount of training data. With 200 instances, EXP-08 reached a weighted F1-score of 93.16%, which already exceeded the PT-P1 baseline of 83.98%. Increasing the dataset to 300 instances further improved the weighted F1-score to 95.54% in several experiments. The final set of experiments used 550 samples, where EXP-19 and EXP-20

reached 97.57%, while EXP-22 achieved the highest weighted F1-score of 98.78%.

This results showed that using more training data improved the model performance of the model. A model with more hidden neurons or longer training does not always produce better results when only a small amount of data is available. An additional dataset gave the neural network more examples from which to learn the different forms of hotel customer feedback. That results also showed the important things of balancing model capacity with enough training data to reduce memorization and improve performance on validation data. Dataset expansion is one of the optimization methods that could be used when hyperparameter tuning was not enough to outperform the PT-P1 model.

D. Connecting Code Implementation to Slide Architecture (Pipeline Mapping)

The neural network pipeline in the code followed a sequence from text preprocessing to model training and classification. The hotel customer feedback was first converted into numerical sequences before being passed to the Keras Sequential model consisting of Embedding, GlobalAveragePooling1D, Dense, Dropout, and Softmax output layers. The Embedding layer changed the numerical word values into vectors for the neural network. By the neural network. The GlobalAveragePooling1D layer then combines the word vectors into a single representation of each customer feedback. This layers helped extract useful information from the customer feedback before the information was passed to the Dense layer for classification. The forward pass and loss calculation happened during the model.fit() training process. For each batch, the input passed through the Embedding, GlobalAveragePooling1D, Dense, Dropout, and output layers to produce class probabilities. The configured sparse categorical cross-entropy loss then measured the difference between the predicted output and the actual class. The backward pass and weight updates also occurred automatically during model.fit(). After the loss was calculated, TensorFlow/Keras computed the gradients through backpropagation. The selected optimizer, such as Adam, used these gradients to update the trainable weights and biases. This process was repeated for the training batches and epochs, helping the model reduce its classification error

V. CONCLUSION

In this exercise, a different approach of the experimental process resulted in training and optimizing a TensorFlow/Keras neural network for classifying hotel customer feedback based on sentiment and intent. The experiments approach different model settings, including learning rate, hidden neurons, dropout rate, batch size, epochs, and optimizer. The training and validation loss curves were also monitored to identify the behavior of each model. The early experiments showed that changing the hyperparameters alone was not enough to reach the target performance when only a small amount of training data was used. Increasing the dataset from 50 to 200, 300, and 550 instances resulted in better validation performance across the later experiments. Among all tested configurations, EXP-22 achieved the highest performance and was selected as the champion model. It reached a validation accuracy and weighted F1-score of 98.78%, exceeding the PT-P1 baseline F1-score of 83.98%. Early Stopping ended the training at epoch 20 out of the maximum 45 epochs, while the close training and validation losses supported its classification as an ideal fit.

REFERENCES

- [1] Y. Wen, Y. Liang, and X. Zhu, "Sentiment analysis of hotel online reviews using the BERT model and ERNIE model—Data from China," *PLoS ONE*, vol. 18, no. 3, Art. no. e0275382, Mar. 2023, doi: 10.1371/journal.pone.0275382.
- [2] Y. Bian, R. Ye, J. Zhang, and X. Yan, "Customer preference identification from hotel online reviews: A neural network based fine-grained sentiment analysis," *Computers & Industrial Engineering*, vol. 172, Art. no. 108648, Oct. 2022, doi: 10.1016/j.cie.2022.108648.
- [3] A. Rame et al., "Generalization in neural networks: A broad survey," *Neurocomputing*, vol. 611, Art. no. 128701, Jan. 2025, doi: 10.1016/j.neucom.2024.128701.